Date: 03/03/2026

CS F363 Compiler Construction Assignment (Phase 1) - Language Design and Lexical Analysis
Project Group-ID: 2

Language Chosen: Lua

Shlok Gudadhe	2023A7PS0041H
Krishna Dhawan	2023A7PS0111H
Amay Deodhar	2023A7PS0155H
Jayditya Kabra	2023A7PS1059H
Pritish Saraf	2023A7PS1104H

Table of Contents

Table of Contents **2**
1. Introduction **4**
2. Tools Used **5**
3. Structure of the Lex Program **5**
4. Regular Expressions Used **6**
4.1 Identifiers 6
4.2 Numeric Literals 6
4.2.1 Decimal Numbers 6
4.2.2 Hexadecimal Numbers 6
4.2.3 Binary Numbers 6
4.3 Boolean Values 7
4.4 Nil 7
4.5 String Literals 7
Double-Quoted Strings 7
Single-Quoted Strings 7
4.6 Comments 7
Single-Line Comments 7
Multi-Line Comments 7
4.7 Keywords 8
4.8 Operators 8
Relational Operators 8
Arithmetic Operators 8
Assignment Operator 8
Concatenation Operator 8
4.9 Delimiters and Punctuation 9
4.10 Whitespace Handling 9
4.11 Lexical Error Rule 9
5. Complete CFG **10**
5.1 Program Structure 10
5.2 Statements 10
5.3 Variable Declaration 10
5.4 Assignment Statement 10
5.5 Function Definition 10
5.6 If Statement 11
5.7 While Loop 11
5.8 Expressions 11

5.8.1 Level 1: Logical Expressions 11
5.8.2 Level 2: Relational Expressions 11
5.8.3 Level 3: Arithmetic (Addition/Subtraction) 12
5.8.4 Level 4: Arithmetic (Multiplication/Division) 12
5.8.5 Level 5: Base Expressions 12
5.9 Function Call 12
5.10 Parameters and Arguments 12
6. Conclusion **13**

1. Introduction

This project implements a Lexical Analyzer for a subset of the programming language Lua using the Lex tool.

The lexer reads a Lua source file and identifies different tokens :

- Keywords
- Identifiers
- Numeric literals (Decimal, Hexadecimal, Binary)
- String literals
- Boolean values
- Nil
- Operators
- Delimiters
- Comments

The output of lexical analysis is written to an output file. Any unrecognized character is reported as a lexical error.

This implementation focuses only on lexical analysis and does not include syntax parsing.

2. Tools Used

- Lex (Flex)
- C Language
- GCC Compiler
- Linux Environment

3. Structure of the Lex Program

The `lexer.l` file consists of:

1. Definition Section
2. Rules Section
3. User Code Section (main function)

The program:

- Takes input file and output file as command-line arguments
- Reads the Lua source code
- Prints tokens into the output file
- Reports lexical errors when invalid characters are encountered

4. Regular Expressions Used

Below are the regular expressions implemented in the lexical analyzer.

4.1 Identifiers

LETTER → [a-zA-Z_]
DIGIT → [0-9]

ID → [a-zA-Z_][a-zA-Z0-9_]*

An identifier must:

- Start with a letter or underscore
- Followed by letters, digits, or underscores

4.2 Numeric Literals

4.2.1 Decimal Numbers

NUMBER → [0-9]+(\.[0-9]+)?
Supports :

- Integers
- Floating-point numbers

4.2.2 Hexadecimal Numbers

HEX_NUMBER → 0[xX][0-9a-fA-F]+

Example:

0x64
0x3E8
0xFFFFFF
The lexer converts hexadecimal values into decimals using `strtol`.

4.2.3 Binary Numbers

BIN_NUMBER → [01]+[bB]

Example:

10110b
110010b

Binary values are converted into decimal using `strtol`.

4.3 Boolean Values

BOOLEAN → (true|false)

Recognized as BOOLEAN tokens.

4.4 Nil

Recognized as a NIL token.

4.5 String Literals

Double-Quoted Strings

\"([^\\\"]|\\.)*\"

Single-Quoted Strings

\'([^\\\']|\\.)*\'

Supports escaped characters inside strings.

4.6 Comments

Single-Line Comments

-- ...

Everything after `--` until the newline is ignored.

Multi-Line Comments

--[[...]]

Handled using Lex start conditions.

All content inside multi-line comments is ignored.

4.7 Keywords

The following keywords are recognized:

- local
- function
- if
- else
- elseif
- then
- end
- for
- while
- do
- and
- or
- not
- in
- break
- return

4.8 Operators

Relational Operators

== ~= <= >= < >

Arithmetic Operators

+ - * /

Assignment Operator

=

Concatenation Operator

..

4.9 Delimiters and Punctuation

( ) { } [ ]

, ; . :

4.10 Whitespace Handling

[\t\r\n]+

Whitespace characters are ignored.

4.11 Lexical Error Rule

.

Any unrecognized character is reported as:

LEXICAL ERROR: <character>

5. Complete CFG

5.1 Program Structure

Program → StatementSequence
StatementSequence → Statement StatementSequence
| ε

5.2 Statements

Statement → LocalDeclaration
| Assignment
| IfStatement
| WhileStatement
| FunctionDefinition
| FunctionCall

5.3 Variable Declaration

LocalDeclaration → 'local' identifier '=' Expr

5.4 Assignment Statement

Assignment → identifier '=' Expr

Assignment → identifier '.' identifier '=' Expr

5.5 Function Definition

Normal function:

FunctionDefinition → 'function' identifier '(' ParameterList ')'
StatementSequence
'end'

Method-style function:

FunctionDefinition → 'function' identifier ':' identifier '(' ParameterList ')'
StatementSequence
'end'

5.6 If Statement

```
IfStatement → 'if' Expr 'then' StatementSequence 'end'
IfStatement → 'if' Expr 'then' StatementSequence
            'else' StatementSequence
            'end'
```

5.7 While Loop

```
WhileStatement → 'while' Expr 'do' StatementSequence 'end'
```

5.8 Expressions

Expressions follow precedence levels.

5.8.1 Level 1: Logical Expressions

```
Expr → Expr 'or' AndExpr
   | AndExpr
AndExpr → AndExpr 'and' RelExpr
      | RelExpr
RelExpr → 'not' RelExpr
      | Comparison
```

5.8.2 Level 2: Relational Expressions

```
Comparison → ArithExpr RelOp ArithExpr
         | ArithExpr
RelOp → '==' | '~=' | '<' | '>' | '<=' | '>='
```

5.8.3 Level 3: Arithmetic (Addition/Subtraction)

```
ArithExpr → ArithExpr '+' Term
        | ArithExpr '-' Term
        | Term
```

5.8.4 Level 4: Arithmetic (Multiplication/Division)

```
Term → Term '*' Factor
   | Term '/' Factor
   | Factor
```

5.8.5 Level 5: Base Expressions

```
Factor → identifier
     | number
     | hex_number
     | bin_number
     | string
     | boolean
     | 'nil'
     | '(' Expr ')'
```

5.9 Function Call

```
FunctionCall → identifier '(' ArgumentList ')'
FunctionCall → identifier ':' identifier '(' ArgumentList ')'
```

5.10 Parameters and Arguments

```
ParameterList → identifier ',' ParameterList
            | identifier
            | ε
ArgumentList → Expr ',' ArgumentList
          | Expr
          | ε
```

6. Conclusion

This project successfully implements a lexical analyzer for a subset of the Lua programming language using Lex. It identifies keywords, identifiers, numeric literals (decimal, hexadecimal, binary), strings, boolean values, nil, operators, delimiters, and comments. The analyzer also detects lexical errors for invalid characters. This implementation demonstrates practical understanding of:

- Regular Expressions
- Tokenization
- Lex tool usage
- Structure of a compiler’s lexical analysis phase